

To-Do Application using REST API

Lab Objectives

After completing this lab, the student will be able to:

- Understand RESTful API concepts
- Perform CRUD operations using HTTP methods
- Integrate Flutter applications with external APIs
- Parse JSON data into Dart objects
- Replace local data storage with remote server data

Tools Required

- Flutter SDK
- Dart SDK
- Android Studio / VS Code
- The Flutter app we used in the previous three experiments.
- Internet connection
- `http` package

In the `pubspec.yaml` file, add the dependency:

```
dependencies:
```

```
# your existing packages
```

```
http:
```

Introduction

In the previous experiment, tasks were stored locally in SQLite.

In this experiment, the application will use a **remote REST API** to store and manage tasks.

This allows the application to:

- Persist data externally
- Share data across devices
- Simulate real-world backend integration

API Information

Base URL: <https://build-wild-moon-3520.fly.dev/tasks>

Endpoints

Method	Endpoint	Description
GET	/tasks	Get all tasks
POST	/tasks	Create new task
GET	/tasks/:id	Get single task
PUT	/tasks/:id	Update task
DELETE	/tasks/:id	Delete task

LAB

Task 1: Update Task Model for API Integration

Objective:

Modify the Task model to support API communication using JSON.

In `models/task.dart`, Add:

○ ○ ○

```
factory Task.fromJson(Map<String, dynamic> json) {
  return Task(
    id: json['id'],
    title: json['title'],
    description: json['description'],
    category: json['category'],
    isCompleted: json['is_done'],
  );
}

Map<String, dynamic> toJson() {
  return {
    "title": title,
    "description": description,
    "category": category,
    "is_done": isCompleted,
  };
}
```

Task 2: Create Remote Tasks Repository

Objective:

Create a repository class responsible for handling all API operations.

repositories/remote_tasks_repository.dart

○ ○ ○

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/task.dart';

class RemoteTasksRepository {

  final String baseUrl = "https://build-wild-moon-3520.fly.dev/tasks";

  /// GET all tasks
  Future<List<Task>> getTasks() async {
    final response = await http.get(Uri.parse(baseUrl));

    if (response.statusCode == 200) {
      List data = jsonDecode(response.body);
      return data.map((e) => Task.fromJson(e)).toList();
    } else {
      throw Exception("Failed to load tasks");
    }
  }
}
```

```

/// POST create task
Future<void> createTask(Task task) async {
  final response = await http.post(
    Uri.parse(baseUrl),
    headers: {"Content-Type": "application/json"},
    body: jsonEncode(task.toJson()),
  );

  if (response.statusCode != 201) {
    throw Exception("Failed to create task");
  }
}

/// DELETE task
Future<void> deleteTask(int id) async {
  final response = await http.delete(
    Uri.parse("$_baseUrl/$id"),
  );

  if (response.statusCode != 200) {
    throw Exception("Failed to delete task");
  }
}

/// PUT update task
Future<void> updateTask(Task task) async {
  final response = await http.put(
    Uri.parse("$_baseUrl/${task.id}"),
    headers: {"Content-Type": "application/json"},
    body: jsonEncode(task.toJson()),
  );

  if (response.statusCode != 200) {
    throw Exception("Failed to update task");
  }
}
}

```

Task 3: Fetch Tasks Using Remote Repository

Objective:

Use the repository to fetch tasks instead of local list.

inside **task_list_screen.dart**

○ ○ ○

```
RemoteTasksRepository _repository = RemoteTasksRepository();
List<Task> _tasks = [];

@override
void initState() {
  super.initState();
  loadTasks();
}

void loadTasks() async {
  _tasks = await _repository.getTasks();
  setState(() {});
}
```

Task 4: Create Task Using Remote Repository

Objective:

Send new tasks to the API instead of saving locally.

inside `add_task_screen.dart`

○ ○ ○

```
RemoteTasksRepository _tasksRepository = RemoteTasksRepository();

void _saveTask() async {

  if (_titleController.text.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text("Please enter a task title")),
    );
    return;
  }

  Task newTask = Task(
    title: _titleController.text,
    description: _descriptionController.text,
    category: _selectedCategory,
  );

  await _tasksRepository.createTask(newTask);
  if(mounted){
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Task added successfully'), backgroundColor: Colors.green,)
    );
  }

  Navigator.pop(context);
}
}
```

Task 5: Delete Task Using Repository

Objective:

Delete tasks from the API instead of local list.

inside `task_list_screen.dart`

```
○ ○ ○

void showDeleteDialog(int index) {
  showDialog(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: Text("Delete Task"),
        content: Text("Are you sure?"),
        actions: [

          TextButton(
            onPressed: () => Navigator.pop(context),
            child: Text("Cancel"),
          ),

          TextButton(
            onPressed: () async {

              await _repository.deleteTask(_tasks[index].id);
              loadTasks();
              setState(() {});
              Navigator.pop(context);
            },
            child: Text("Delete"),
          ),
        ],
      );
    },
  );
}
```

POST-LAB

1. Implement the Update operation on Tasks using API
2. Use the Abstraction and Interface Pattern to modify the local tasks repo and remote tasks repo with one abstract interface
3. Handle API errors (show message if request fails)